

Seguridade

Miguel Rodríguez Penabad

Laboratorio de Bases de Datos

Universidade da Coruña



UNIVERSIDADE DA CORUÑA

6 de marzo do 2025



- 1 Introducción
- 2 Autenticación e autorización
- 3 Control de acceso discrecional
- 4 Caso particular: Oracle
- 5 Técnicas adicionais
- 6 Auditoría

Introdución

A seguridade da información é algo fundamental para calquera organización.

Normalmente céntrase en garantir:

- **Confidencialidade:** Existen datos privados que deben estar protexidos ante accesos non autorizados.
- **Dispoñibilidade:** Non deben producirse denegacións de acceso a datos sobre os que hai dereito de acceso.
- **Integridade:** Non deben producirse modificacións non adecuadas, nin danos na información.

Desde un punto de vista da aplicación de normas de seguridade, esta verase influenciada por:

- Lexislación vixente.
Aspectos legais, como o dereito a dispoñer de determinada información, confidencialidade etc.
As leis como a LOPD (Lei Orgánica de Protección de datos) e máis recentemente o RXPd (Regulamento Xeral de Protección de Datos) son de obrigado cumprimento.
- Normas específicas da organización.
Pode clasificarse a información en distintos niveis de seguridade.
Isto determina que (grupos de) usuarios teñen acceso sobre cada dato, e que poden facer sobre el (as accións permitidas).

A seguridade da información pode estar vinculada a elementos de moi diversa índole:

- Edificios (seguridade perimetral).
- Persoal.
- Dispositivos, ou hardware en xeral.
- Sistemas operativos.
- Comunicacións.
- Software en xeral.
- **Bases de datos.**

Introdución

ISO/IEC 27001

ISO/IEC 27001 é un estándar da seguridade da información que permite establecer os requisitos necesarios para crear, manter e mellorar un Sistema de Xestión de Seguridade da Información (SXI) dunha organización.

Establece un conxunto de boas prácticas para permitir ás organizacións realizar unha correcta xestión dos riscos e a aplicación de procedementos e controis necesarios para contrarrestar, ou incluso eliminar, eses riscos.

A análise de riscos da organización estará baseada nun inventario de activos, un inventario de ameazas, e a definición dun nivel de risco aceptable para a organización.

Propoñerá unha serie de plans de accións para contrarrestar eses riscos.

Exemplo:

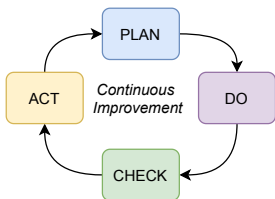
- *Activo*: Base de datos de clientes
 - Confidencialidade: Moi Alta
 - Dispoñibilidade: Alta
 - Integridade: Moi alta.
- *Ameaza*: Divulgación de información confidencial de clientes
 - Probabilidade: Baixa
 - Impacto: Moi alto
 - Risco: Alto
- *Nivel de Risco* aceptable pola organización: Medio
- Plans de acción:

...

Introdución

ISO/IEC 27001

Está baseada normalmente no ciclo de Deming ou PDCA (Plan-Do-Check-Act):



- **Plan:** Establecer obxectivos e procesos, e facer unha planificación temporal.
- **Do:** Implementar eses obxectivos e procesos.
- **Check:** Verificar o éxito ou fracaso das medidas tomadas.
- **Act** (a veces *Adjust*): Recopilar o aprendido na fase anterior e poñelo en marcha. A veces aparecen recomendacións que iniciarían un novo ciclo.

Exemplo: Ante a ameaza de perder a BD cos datos dos clientes:

1. *Plan:* Realizar copias de seguridade.
Copia completa cada mes, incremental diaria.
Realizar 2 probas anuais.
2. *Do:* Implementar os backups da base de datos.
3. *Check:* Verificar que as copias se realizan diariamente/mensualmente, e comprobar 2 veces ó ano se poden ser restauradas correctamente.
4. *Act:* Comprobando o resultado do *check* do punto anterior, decidir se hai que axustar algunha acción.

Introducción

Seguridade nas bases de datos

A **Seguridade** impleméntase nos SXBD para protexer a BD de **ameazas** e **ataques**, sexan intencionados ou accidentais.

Os ataques ou accesos indebidos poden producir perda de:

- Confidencialidade
- Dispoñibilidade.
- Integridade (incluíndo roubo ou fraude).

Podemos implementar **medidas** para contrarrestar ou eliminar os riscos:

- Control de acceso: Autenticación e autorización.
- Control de Acceso Obrigatorio.
- Control de Acceso Discrecional.
- Outras medidas:
 - BD estatísticas.
 - Cifrado de datos.
 - Auditoría.
 - Etc.

- 1 Introducción
- 2 Autenticación e autorización
- 3 Control de acceso discrecional
- 4 Caso particular: Oracle
- 5 Técnicas adicionais
- 6 Auditoría

Autenticación e autorización

AuthIDs

- Un *Authorization identifier* (*AuthID*) pode ser:
 - Un identificador ou nome de usuario.
 - Un nome dun rol (falaremos deste concepto máis adiante).
 - PUBLIC é un *pseudo-AuthID* que referencia a totalidade dos AuthIDs (presentes e futuros) da BD.
- Identificador (conta, nome) de usuario
 - É un identificador de SQL que se utiliza para conectarse á base de datos.
 - Conceptualmente identifica un usuario, persoa ou programa que accede á base de datos.
 - O estándar non especifica como crear estes nomes de usuarios.
 - Exemplos de varios SXBD:

Oracle:

```
-- Como identificador, se hai caracteres como o punto,  
-- o usuario e a clave deben ir entre comiñas dobres  
CREATE USER "nome.usuario" IDENTIFIED BY "contra..sinal";  
-- Identificación externa: SO ou LDAP  
CREATE USER usuario IDENTIFIED EXTERNALLY;
```

PostgreSQL:

```
-- O usuario debe ir entre comiñas dobres se ten caracteres especiais  
-- Pero a clave debe ir SEMPRE entre comiñas simples, como un texto  
CREATE USER "nome.usuario" WITH PASSWORD 'contrasinal';
```

SQL Server:

```
-- Usuario da BD  
CREATE USER usuario WITH PASSWORD='contrasinal';  
-- Usuario baseado en login  
CREATE USER usuario FROM LOGIN usuariowindows;
```


Autenticación e autorización

Autenticación

- A autenticación é o primeiro proceso que realiza o SXBD para realizar o control de acceso dos usuarios á base de datos.
- Consiste en identificar a un usuario e verificar a súa identidade.
- O proceso implica:
 1. Identificarse: o usuario debe indicar o seu *AuthID* (nome ou identificador de usuario).
 2. Debe verificarse, con información adicional, que o usuario é quen di ser.
Isto faise normalmente utilizando unha contrasinal. A verificación pode ser:
 - Realizada completamente polo SXBD (que almacena usuarios coas súas contrasinais –cifradas–).
 - Delegando a verificación no Sistema Operativo.
 - Delegando a verificación noutro sistema externo (por exemplo, un LDAP).(Nota: outros mecanismos, tales como identificación biométrica, da conducta do usuario, ou dispositivos hardware adicionais, poderían ser posibles, pero non os imos considerar neste documento.)
- Unha vez o usuario está autenticado, pasaríamos á segunda fase: a autorización, que indica o que o usuario pode facer.

Autenticación e autorización

Autorización

- A autorización especifica o que un usuario (en xeral, un *authID*) pode facer na base de datos.
- Debemos ter en conta que o feito de que un usuario se autentique correctamente, non ten por que darlle dereito a nada, nin siquiera a conectarse á BD.

Por exemplo, se creamos un usuario en Oracle, este non terá –nunha instalación habitual– o privilexio requerido para conectarse á BD.

- Existen fundamentalmente 2 tipos de mecanismos de control de acceso:
 - Control de Acceso Obrigatorio (*MAC: Mandatory Access Control*).
 - O estándar SQL non inclúe soporte para este control de acceso.
 - O nome pode ser enganoso, xa que non é o máis habitualmente implementado nos SXBD.
 - Control de Acceso Discrecional (*DAC: Discretionary Access Control*).
 - Está definido no estándar SQL.
 - Está baseado na concesión e revocación de privilexios, utilizando as sentencias GRANT e REVOKE.
 - Normalmente inclúe a autorización baseada en roles.

Control de Acceso Obrigatorio

(MAC: Mandatory Access Control)

- Está baseado en políticas a nivel de sistema (non modificables por usuarios individuais).
 - A cada obxecto da base de datos asígnaselle unha *clase de seguridade*.
 - A cada usuario asígnaselle un *nivel de autorización* para cada clase de seguridade.
 - Un usuario pode ler ou escribir un obxecto baseándose nos niveis de autorización e clases de seguridade.
- Trata de garantir que os datos confidenciais non pasen a un usuario sen o nivel de seguridade axeitado.
- Exemplo: Modelo Bell-LaPadula
 - Define Clases de Seguridade. Por exemplo:

<i>TS:</i>	<i>Top Secret</i>
<i>S:</i>	<i>Secret</i>
<i>C:</i>	<i>Confidential</i>
<i>U:</i>	<i>Unclassified</i>

- Asigna unha clases de seguridade a obxectos e suxeitos:
 - $Class(O)$: Clase de seguridade dun Obxecto O (táboa, tupla, ...)
 - $Class(S)$: Clase de seguridade dun Suxeito S (usuario, programa, ...)
- Define 2 regras ou *propiedades de seguridade*:
 - **Simple**: Un Suxeito S pode ler un Obxecto O se $Class(S) \geq Class(O)$.
 - **Estrela (*)**: Un Suxeito S pode escribir un Obxecto O se $Class(S) \leq Class(O)$.
 - Permite o fluxo de información de niveis baixos a altos.
 - Non permite o fluxo de niveis altos a baixos (que podería comprometer a seguridade).

- 1 Introducción
- 2 Autenticación e autorización
- 3 Control de acceso discrecional**
- 4 Caso particular: Oracle
- 5 Técnicas adicionales
- 6 Auditoría

Control de Acceso Discrecional

Privilexios

O control de acceso discrecional está baseado na concesión e revocación de privilexios.

Existen 2 tipos de privilexios: a nivel de sistema e a nivel de obxecto.

Privilexios de sistema ou de servidor:

- Especifican accións que un usuario pode levar a cabo, sen estar vinculadas con ningún obxecto en concreto. Ex: crear táboas.
- Non está definido no SQL estándar (queda aberto á implementación), pero practicamente todos os SXBD os inclúen.
- Normalmente, a xestión é sintacticamente igual á xestión de roles.

Exemplos (Oracle):

```
CREATE TABLE           -- Crear táboas no esquema propio
CREATE ANY TABLE       -- Crear táboas en calquera esquema
SELECT ANY              -- Seleccionar datos de calquera táboa en calquera esquema
AUDIT ANY               -- Auditar calquera obxecto do sistema
CREATE MATERIALIZED VIEW -- Crear vistas materializadas no esquema propio

-- Poden consultarse en SYSTEM_PRIVILEGE_MAP
-- Nunha instalación de Oracle 19c típica hai 257 privilexios de sistema
```

Control de Acceso Discrecional

Privilexios

- **Privilexios a nivel de obxecto:**

Están asociados a un obxecto en concreto, e dependerán do tipo de obxecto

Están definidos no SQL estándar (aínda que os xestores utilizan habitualmente privilexios adicionais)

Para táboas e vistas:

```
SELECT [<lista de columnas>]
INSERT [<lista de columnas>]
DELETE
UPDATE [<lista de columnas>]
REFERENCES [<lista de columnas>]
TRIGGER [<lista de columnas>]
```

Existen outros, como USAGE (para dominios) ou EXECUTE (para procedementos almacenados).

O creador dun obxecto ten todos os privilexios sobre ese obxecto e a potestade de pasar eses privilexios a outros.

A política de seguridade de SQL indica que non debemos poder inferir información á que non temos acceso. Se un usuario pretende seleccionar datos dunha táboa á que non ten acceso, o SXBD debería indicar que tal táboa non existe.

Control de Acceso Discrecional

Concesión de privilexios sobre obxectos da BD

Utilízase a sentencia GRANT:

```
GRANT { <lista_privilexios> | ALL PRIVILEGES }  
  ON <obxecto>  
  TO <lista_authids>  
  [WITH GRANT OPTION]
```

- Un usuario só pode conceder un privilexio sobre o <obxecto> se ten ese privilexio e a potestade de pasalo a outros.
- O creador dun obxecto ten todos os privilexios sobre el, e a potestade de pasalos a outros.
- A sentencia GRANT permite conceder varios privilexios á vez, pero sobre un único <obxecto>.
- ALL PRIVILEGES é a lista de todos os privilexios aplicables ó tipo de obxecto implicado.
- <lista_authids> é unha lista (separada por comas) de AuthIDs (que poden ser usuarios, roles ou o pseudo-authID PUBLIC).
- Utilizando WITH GRANT OPTION concédese tamén a potestade de pasar ese privilexio a outros.

Exemplos:

```
GRANT SELECT, DELETE, INSERT ON tab1 TO usuario1;  
GRANT UPDATE(sal) ON emp TO scott, role345 WITH GRANT OPTION;  
GRANT ALL PRIVILEGES ON EMP TO theboss;  
GRANT SELECT ON emp TO PUBLIC; -- Todos os usuarios, incluso os creados no futuro, poderán seleccionar de emp
```

Control de Acceso Discrecional

Revocación de privilegios sobre obxectos da BD

Utilízase a sentencia REVOKE:

```
REVOKE [GRANT OPTION FOR] { <lista_privilexios> | ALL PRIVILEGES }  
  ON <obxecto>  
  FROM <lista_authids>  
  { CASCADE | RESTRICT }
```

- Un usuario só pode revocar un privilexio que concedeu explicitamente cunha sentenza GRANT previa.
- Usando GRANT OPTION FOR só se retira a potestade para pasar os privilexios a outros (pero os privilexios consérvanse).
- Se algún usuario da <lista_authids> propagou algún privilexio a outros:
 - CASCADE retira o privilexio en cascada a todos os Authlds implicados.
 - RESTRICT produce un erro e non revoca os privilexios.

Exemplos:

```
REVOKE DELETE, INSERT ON tab1 FROM usuario1;  
REVOKE UPDATE(sal) ON emp FROM scott CASCADE;  
REVOKE SELECT ON emp FROM PUBLIC;
```


Control de Acceso Discrecional

Liñas e cadeas de concesión de privilexios

Ó executar unha sentenza GRANT concedendo un privilexio a un AuthID establécese unha liña de concesión.

Cando se propaga un privilexio a terceiros, poden crearse complexas cadeas de concesión.

Un AuthID perde un privilexio se se lle retira por todas as liñas de concesión.

-- u1 é o propietario dunha táboa xenérica "t"

1. (u1) GRANT SELECT ON t to u2, u3
WITH GRANT OPTION;

-- u2 dá permisos a u2 e u3, coa opción grant.

2. (u2) GRANT SELECT ON t to u4;

3. (u3) GRANT SELECT ON t to u4;

-- Tanto u2 como u3 poden dar o permiso a u3.

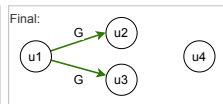
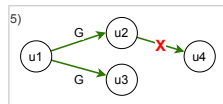
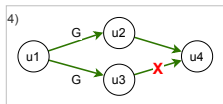
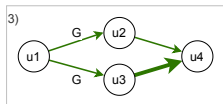
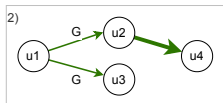
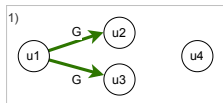
-- Cada sentenza GRANT dá lugar a unha liña de concesión.

4. (u3) REVOKE SELECT ON t FROM u4;

-- u3 revoca a súa liña de concesión, pero
-- u4 mantén o privilexio debido á liña de
-- concesión desde u2.

5. (u2) REVOKE SELECT ON t FROM u4;

-- u2 revoca a súa liña de concesión.
-- Agora u4 perde o privilexio.



Control de Acceso Discrecional

Liñas e cadeas de concesión de privilexios

-- Os puntos 1-3 son iguais á transparencia anterior

1. (u1) GRANT SELECT ON t to u2, u3

WITH GRANT OPTION;

2. (u2) GRANT SELECT ON t to u4;

3. (u3) GRANT SELECT ON t to u4;

4. (u1) REVOKE SELECT ON t FROM u3 CASCADE;

-- u1 revoca o privilexio a u3 coa opción en cascada.

-- u3 perde o privilexio, e a súa liña de concesión

-- a u4 desaparece.

5. (u1) REVOKE GRANT OPTION FOR SELECT ON t

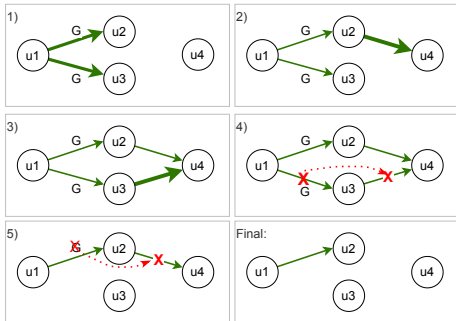
FROM u2 CASCADE;

-- u1 revoca a "grant option" a u2 en cascada.

-- u2 mantén o privilexio pero perde a potestade de

-- pasalo. A súa liña de concesión a u4 desaparece.

-- Agora u4 perde o privilexio.



Control de Acceso Discrecional

Roles

Un rol é un tipo especial de AuthID que se utiliza para facilitar a xestión dos privilexios.

Características dun rol:

- É un AuthID que non pode conectarse á BD (non é un usuario).
- Podemos concederlle privilexios (ou outros roles).
- Podemos asignar ("conceder", `grant`) o rol a outros AuthIDs (usuarios, roles, `PUBLIC`), co que pasan a ter os privilexios do rol concedido.

Exemplo:

```
CREATE ROLE facturador;
CREATE ROLE xestor;

GRANT ALL PRIVILEGES ON factura TO facturador;
GRANT SELECT ON artigo TO facturador;

GRANT ALL PRIVILEGES ON artigo TO xestor;
GRANT facturador TO xestor;

GRANT facturador TO facturador1, facturador2;
GRANT xestor TO xestor1;
REVOKE facturador FROM facturador2;
REVOKE REFERENCES ON factura FROM facturador;

DROP ROLE xestor;
```

Control de Acceso Discrecional

Roles

A concesión e revocación de roles utiliza tamén sentenzas GRANT e REVOKE.

```
GRANT <rol> [, <rol>, ... ]  
    TO <lista_authids>  
    [WITH ADMIN OPTION]
```

```
REVOKE [ADMIN OPTION FOR] <rol> [, <rol>, ... ]  
    FROM <lista_authids>
```

Para os roles, estas sentenzas GRANT e REVOKE non inclúen ningunha cláusula ON <obxecto>, e ademais utilizan opcionalmente a cláusula WITH ADMIN OPTION.

O funcionamento desta cláusula non é exactamente igual a with grant option.

Neste caso, ademais do rol pásase a potestade de *administrar* ese rol:

- Permite conceder privilexios ou outros roles a ese rol.
- Permite conceder o rol, ou revocalo, a calquera lista de AuthIDs.
- Permite eliminar (drop) ese rol.

Control de Acceso Discrecional

Roles

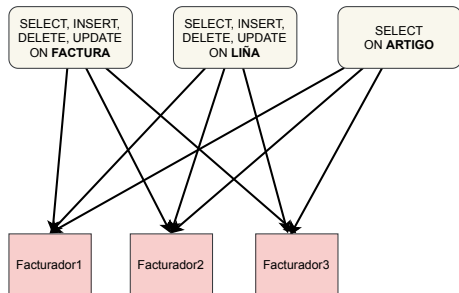
Ventajas do uso de roles (exemplo):

Situación inicial: Temos 3 usuarios (facturador1, facturador2, facturador3) que necesitan xestionar facturas. Para iso necesitan poder seleccionar, insertar, borrar e modificar nas táboas Factura e Liña (Liña no script SQL), e poder seleccionar da táboa Artigo.

A posteriori faremos o seguinte:

- Engadimos un novo facturador: facturador4.
- Engadimos un xestor (xestor1) que pode xestionar facturas e tamén modificar o prezo dos artigos.

```
GRANT SELECT, INSERT, DELETE, UPDATE
  ON Factura
  TO facturador1, facturador2, facturador3;
GRANT SELECT, INSERT, DELETE, UPDATE
  ON Lina
  TO facturador1, facturador2, facturador3;
GRANT SELECT
  ON Artigo
  TO facturador1, facturador2, facturador3;
```



Control de Acceso Discrecional

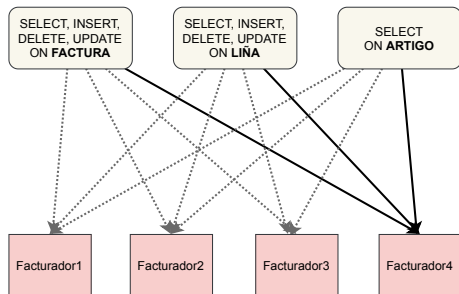
Roles

Ventajas do uso de roles (exemplo):

Engadimos un novo facturador: facturador4.

Sen usar roles, temos que daros permisos en cada unha das tres táboas a facturador4.

```
GRANT SELECT, INSERT, DELETE, UPDATE
  ON Factura
  TO facturador4;
GRANT SELECT, INSERT, DELETE, UPDATE
  ON Lina
  TO facturador4;
GRANT SELECT
  ON Artigo
  TO facturador4;
```



Control de Acceso Discrecional

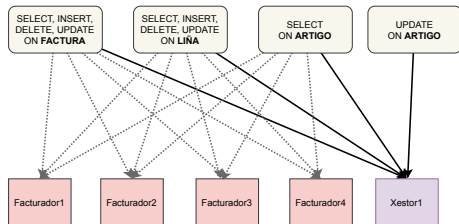
Roles

Ventajas do uso de roles (exemplo):

Engadimos un xestor (xestor1) que pode xestionar facturas e tamén modificar o prezo dos artigos.

De novo, sen usar roles, temos que daros permisos en cada unha das tres táboas (no script úsase unha soa sentenza SQL para os 2 permisos sobre Artigo).

```
GRANT SELECT, INSERT, DELETE, UPDATE
  ON Factura
  TO xestor1;
GRANT SELECT, INSERT, DELETE, UPDATE
  ON Lina
  TO xestor1;
GRANT SELECT, UPDATE(prezo)
  ON Artigo
  TO xestor1;
```



Control de Acceso Discrecional

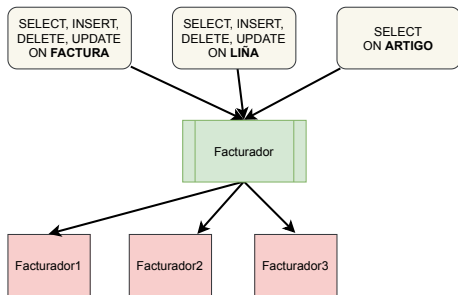
Roles

Ventajas do uso de roles (exemplo):

Volvamos á situación inicial, pero agora **usando roles**.

Inicalmente, pode parecer máis laborioso (hai máis sentenzas SQL) ...

```
CREATE ROLE facturador;  
GRANT SELECT, INSERT, DELETE, UPDATE  
  ON Factura  
  TO facturador;  
GRANT SELECT, INSERT, DELETE, UPDATE  
  ON Lina  
  TO facturador;  
GRANT SELECT  
  ON Artigo  
  TO facturador;  
GRANT facturador  
  TO facturador1, facturador2, facturador3;
```



Control de Acceso Discrecional

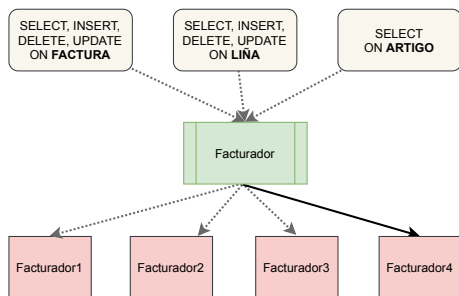
Roles

Ventajas do uso de roles (exemplo):

Engadimos un novo facturador: facturador4.

Só temos que engadir ese usuario rol facturador (concederlle ese rol ó usuario).

```
GRANT facturador  
  TO facturador4;
```



Control de Acceso Discrecional

Roles

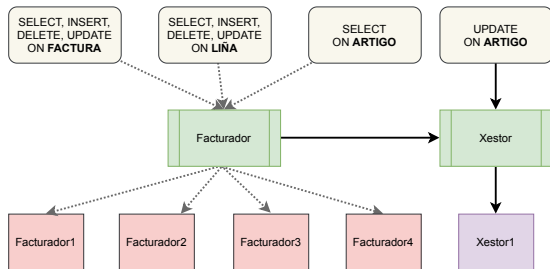
Ventajas do uso de roles (exemplo):

Engadimos un xestor (xestor1) que pode xestionar facturas e tamén modificar o prezo dos artigos.

En previsión de que no futuro se engadan máis xestores, creamos un rol xestor. Engadimos os privilexios necesarios outorgando o rol facturador máis o privilexio independente para actualizar o prezo do Artigo, e asignamos o rol xestor a xestor1.

Se no futuro necesitase outro usuario xestor2, só teríamos que asignarlle o rol xestor.

```
CREATE ROLE xestor;  
GRANT facturador  
    TO xestor;  
GRANT UPDATE(prezo)  
    ON Artigo  
    TO xestor;  
GRANT xestor  
    TO xestor1;
```



- 1 Introducción
- 2 Autenticación e autorización
- 3 Control de acceso discrecional
- 4 Caso particular: Oracle**
- 5 Técnicas adicionais
- 6 Auditoría

Seguridade en Oracle

Privilexios sobre obxectos

Oracle sigue razoablemente ben o estándar SQL no control de acceso discrecional, aínda que ten algunhas particularidades.

Inclúe permisos adicionais para diversos tipos de obxecto.

Entre eles: ALTER, AUDIT, INDEX, ...

Poden verse todos (hai 26) en TABLE_PRIVILEGE_MAP.

A sentencia GRANT é igual á especificada polo estándar, salvo porque a palabra PRIVILEGES é opcional depois de ALL (ó igual que na sentencia REVOKE).

```
GRANT { <lista_privilexios> | ALL [PRIVILEGES] }  
      ON <obxecto>  
      TO <lista_authids>  
      [WITH GRANT OPTION]
```

A sentencia REVOKE non inclúe a especificación CASCADE ou RESTRICT para definir o comportamento no caso de que os privilexios se propagaran a terceiros. O comportamento de Oracle é en cascada.

Tampouco permite revocar só a potestade de pasarllo a outros (GRANT OPTION FOR), debe eliminar por completo o privilexio.

```
REVOKE { <lista_privilexios> | ALL [PRIVILEGES] }  
       ON <obxecto>  
       FROM <lista_authids>
```

Seguridade en Oracle

Privilexios de sistema e Roles

O estándar SQL non especifica nada sobre privilexios do sistema (queda aberto a implementacións).

Oracle utiliza privilexios do sistema e roles, tendo unha serie de roles predeterminados.

A xestión (grant/revoke) é idéntica, pero hai unha diferenza no comportamento: os privilexios toman efecto de forma inmediata, mentras que os roles hai que activalos con SET ROLE <rol>, ... ou reconectando.

Ex. de Privilexios do sistema

```
CREATE TABLE      -- Crear táboas no esquema propio
CREATE ANY TABLE  -- Crear táboas en calquera esquema
CREATE USER        -- Crear usuarios
DELETE ANY TABLE  -- Borrar filas de calquera táboa
CREATE TABLESPACE -- Crear un espazo para táboas
-- Todos os privilexios de sistema en SYSTEM_PRIVILEGE_MAP
```

Ex. de Roles predeterminados:

```
CONNECT  -- Conectarse (privilexios CREATE SESSION e SET CONTAINER)
RESOURCE -- Crear elementos (táboas, secuencias, ... no esquema propio)
DBA      -- Tarefas de DBA (Administrador da Base de Datos) desde "dentro":
          -- Xestiona usuarios, táboas, ... pero non pode p.ex. montar a BD
AUDIT_ADMIN -- Configurar a auditoría do sistema
```

Seguridade en Oracle

Privilexios de sistema e Roles

A concesión e revocación de e roles en Oracle sigue basicamente o estándar. Utiliza a mesma sintaxe para xestionar os privilexios do sistema.

Oracle non permite revokar só a ADMIN OPTION FOR, debe revocarse o rol ou privilexio por completo.

```
GRANT { <privilexio_sistema> | <rol> }, ...  
  TO <lista_authids>  
  [WITH ADMIN OPTION]
```

```
REVOKE { <privilexio_sistema> | <rol> }, ...  
  FROM <lista_authids>
```

```
[DBA] CREATE ROLE rol1;
```

```
-- usr poderá administrar o rol "rol1"
```

```
[DBA] GRANT rol1 TO usr WITH ADMIN OPTION;
```

```
-- usr2 poderá administrar o rol "rol1"
```

```
[USR] GRANT rol1 TO usr2 WITH ADMIN OPTION;
```

```
-- usr2 concede o rol a outros usuarios...
```

```
[USR2] GRANT rol1 TO usr3, usr4;
```

```
-- usr revoca o rol de usuarios... incluso de usr, que foi quen llo concedeu!
```

```
[USR2] REVOKE rol1 FROM usr4, usr;
```

Seguridade en Oracle

Privilexios de sistema e Roles

Veremos a través de exemplos:

- Que crear un usuario non basta para que teña acceso á BD.
- Os privilexios asociados a un rol non se asignan a un usuario ata que o usuario activa ese rol.
- Que revocar un rol non fai que o usuario perda inmediatamente os privilexios asociados.
- Que os privilexios (sobre obxectos ou a nivel do sistema) teñen un efecto inmediato.
- Que Oracle evita a inferencia de información sobre elementos sobre os que non ten privilexios.

```
-- Creamos un usuario
```

```
[DBA] CREATE USER usr IDENTIFIED BY "key" QUOTA UNLIMITED ON users;
```

```
[USR] CONNECT usr/key
```

```
-- ERRO: USR non ten o privilexio CREATE SESSION
```

```
[DBA] GRANT CONNECT TO usr; -- CONNECT é un rol que contén o privilexio CREATE SESSION
```

```
[USR] CONNECT usr/key -- Conectado (activou o rol CONNECT no momento da conexión)
```

Seguridade en Oracle

Privilexios de sistema e Roles

Os privilexios asociados a un rol non se (des)asignan a un usuario ata que o usuario o activa (ou intenta), pero se borramos o rol si que se eliminan os privilexios inmediatamente.

```
[DBA] CREATE ROLE creador;  
[DBA] GRANT CREATE TABLE TO creador;
```

```
[DBA] GRANT creador TO usr;  
[USR] CREATE TABLE T(x int CONSTRAINT pk_t PRIMARY KEY); -- Fallo: privilexios insuficientes  
[USR] SET ROLE creador; -- SET ROLE ALL ou CONNECT usr/key tamén servirían: activarían todos  
[USR] CREATE TABLE T(x int CONSTRAINT pk_t PRIMARY KEY); -- Táboa creada
```

-- VERSIÓN 1: REVOCAR o rol creador

```
[DBA] REVOKE creador FROM usr;  
[USR] CREATE TABLE T2(x int CONSTRAINT pk_t2 PRIMARY KEY); -- Táboa creada  
[USR] SET ROLE ALL; -- Tamén CONNECT usr/key: Activa todos os roles, pero xa non ten "creador"  
[USR] CREATE TABLE T3(x int CONSTRAINT pk_t3 PRIMARY KEY); -- Fallo: privilexios insuficientes
```

-- VERSIÓN 2: ELIMINAR o rol creador

```
[DBA] DROP ROLE creador;  
[USR] CREATE TABLE T2(x int CONSTRAINT pk_t2 PRIMARY KEY); -- Fallo: privilexios insuficientes
```


Seguridade en Oracle

Privilexios de sistema e Roles

Os privilexios (tanto de obxecto como de sistema) toman efecto de xeito inmediato.

```
[USR] CREATE TABLE T(x int CONSTRAINT pk_t PRIMARY KEY); -- Fallo: privilexios insuficientes
[DBA] GRANT CREATE TABLE TO usr;
[USR] CREATE TABLE T(x int CONSTRAINT pk_t PRIMARY KEY); -- Táboa creada

[DBA] REVOKE CREATE TABLE FROM usr;
[USR] CREATE TABLE T2(x int CONSTRAINT pk_t2 PRIMARY KEY); -- Fallo: privilexios insuficientes
```

Vemos tamén que Oracle sigue o estándar no referente a non poder inferir información á que non se ten acceso: Se USR non ten acceso á táboa SCOTT.EMP o erro obtido é “a táboa ou vista non existe”.

```
[USR] SELECT * FROM SCOTT.EMP; -- ERROR en línea 1: ORA-00942: la tabla o vista no existe
[DBA] GRANT SELECT ON SCOTT.EMP TO usr;
[USR] SELECT * FROM SCOTT.EMP; -- Obtén os datos da táboa
```

Seguridade en Oracle

O catálogo

Oracle non sigue o estándar SQL no que ó catálogo do sistema se refire.

Principais vistas do catálogo relacionadas coa seguridade:

- Globais (nomes de privilexios)

TABLE_PRIVILEGE_MAP: Privilexios sobre obxectos (todos, non só táboas).

SYSTEM_PRIVILEGE_MAP: Privilexios a nivel de sistema.

- Privilexios sobre obxectos:

USER_TAB_PRIVS: Obxectos con privilexios (propietario, *grantor* ou *grantee*).

USER_TAB_PRIVS_MADE: Obxectos dos que o usuario é propietario e sobre os que concedeu privilexios.

USER_TAB_PRIVS_RECD: Obxectos sobre os que o usuario recibiu privilexios.

- Privilexios de sistema:

USER_SYS_PRIVS: Privilexios de sistema concedidos ó usuario actual.

ROLE_SYS_PRIVS: Privilexios sobre obxectos concedidos a roles ós que o usuario ten acceso.

DBA_SYS_PRIVS: Todos os privilexios do sistema concedidos a usuarios ou roles.

- Sobre roles:

ROLE_TAB_PRIVS: Privilexios sobre obxectos concedidos a roles ós que o usuario ten acceso.

ROLE_SYS_PRIVS: Privilexios do sistema concedidos a roles ós que o usuario ten acceso.

ROLE_ROLE_PRIVS: Roles concedidos a roles ós que o usuario ten acceso.

USER_ROLE_PRIVS: Roles concedidos ó usuario.

DBA_ROLES: Todos os roles.

Por suposto, dependendo do usuario que execute a consulta (dos seus privilexios), poderá ver ou non algunhas das vistas do catálogo, e o contido poderá variar.

Seguridade en Oracle

Exercicio 1

Considera a seguinte secuencia de sentencias.

Para cada unha antepónse o usuario que a executa. (U1 é o creador da táboa T)

- 1 (U1) `grant select on T to U2 with grant option;`
- 2 (U2) `grant select on U1.T to U3 with grant option;`
- 3 (U1) `revoke select on T from U3;`
- 4 (U3) `select * from U1.T;`

- a) A sentencia 2 falla.
- b) A sentencia 3 falla.
- c) A sentencia 4 falla.

Seguridade en Oracle

Exercicio 2

Sexan r1 e r2 dous roles. Considera a seguinte secuencia de sentencias:

1. `grant r1 to r2`
2. `grant r1 to usuario`
3. `grant r2 to usuario`
4. `revoke r1 from usuario`

- a) O usuario mantén os permisos do rol r1.
- b) O usuario perde os permisos do rol r1.
- c) A primeira sentencia falla, xa que non se pode facer `grant` dun role a outro role.

Seguridade en Oracle

Exercicio 3

Examina a seguinte secuencia de sentencias. Indica cal é a primeira sentencia que falla.

1. (DBA) CREATE USER usuario IDENTIFIED BY "clave";
2. (DBA) GRANT CONNECT TO usuario;
3. (DBA) GRANT SELECT ON taboa TO usuario;
4. (usuario) CONNECT usuario/clave;
5. (DBA) REVOKE CONNECT FROM usuario;
6. (usuario) SELECT * FROM DBA.taboa;
7. (DBA) REVOKE SELECT ON taboa FROM usuario;
8. (usuario) SELECT * FROM DBA.taboa;
9. (usuario) CONNECT usuario/clave;

- a) Falla a sentencia 6 porque o usuario se desconecta ó perder o permiso de conexión.
- b) Falla a sentencia 8 porque o usuario xa non pode acceder á táboa.
- c) Falla a sentencia 9 porque o usuario perdeu o permiso de conexión.

- 1 Introducción
- 2 Autenticación e autorización
- 3 Control de acceso discrecional
- 4 Caso particular: Oracle
- 5 Técnicas adicionais**
- 6 Auditoría

Técnicas adicionais de seguridade

Ademais co control de acceso obrigatorio e discrecional, extendido mediante o uso de roles, podemos aplicar outras técnicas que melloran a seguridade nas nosas bases de datos.

Enre elas:

- Vistas (+ control de acceso discrecional).
- Bases de datos estatísticas.
- Cifrado da información.
- Prevención de inxección SQL.
- Copias de seguridade.
- Auditoría.

Técnicas adicionais de seguridade

Vistas como elemento de seguridade

É habitual querer limitar o acceso a certas filas ou certas columnas dunha táboa.

Non sempre é posible dar os permisos directamente sobre a táboa

```
SELECT EMPNO, ENAME, JOB, MGR, HIREDATE  
FROM EMP
```

empno	ename	job	mgr	hiredate	sal	comm	deptno
7.839	KING	PRESIDENT	[NULL]	1981-11-17	5.000	[NULL]	10
7.782	CLARK	MANAGER	7.839	1981-06-09	2.450	[NULL]	10
7.934	MILLER	CLERK	7.782	1982-01-23	1.300	[NULL]	10
7.566	JONES	MANAGER	7.839	1981-04-02	2.975	[NULL]	20
7.902	FORD	ANALYST	7.566	1981-12-03	3.000	[NULL]	20
7.369	SMITH	CLERK	7.902	1980-12-17	800	[NULL]	20
7.788	SCOTT	ANALYST	7.566	1987-04-19	3.000	[NULL]	20
7.876	ADAMS	CLERK	7.788	1987-05-23	1.100	[NULL]	20
7.698	BLAKE	MANAGER	7.839	1981-05-01	2.850	[NULL]	30
7.499	ALLEN	SALESMAN	7.698	1981-02-20	1.600	300	30
7.521	WARD	SALESMAN	7.698	1981-02-22	1.250	500	30
7.654	MARTIN	SALESMAN	7.698	1981-09-28	1.250	1.400	30
7.844	TURNER	SALESMAN	7.698	1981-09-08	1.500	0	30
7.900	JAMES	CLERK	7.698	1981-12-03	950	[NULL]	30

```
SELECT *  
FROM EMP  
WHERE DEPTNO=20
```

empno	ename	job	mgr	hiredate	sal	comm	deptno
7.839	KING	PRESIDENT	[NULL]	1981-11-17	5.000	[NULL]	10
7.782	CLARK	MANAGER	7.839	1981-06-09	2.450	[NULL]	10
7.934	MILLER	CLERK	7.782	1982-01-23	1.300	[NULL]	10
7.566	JONES	MANAGER	7.839	1981-04-02	2.975	[NULL]	20
7.902	FORD	ANALYST	7.566	1981-12-03	3.000	[NULL]	20
7.369	SMITH	CLERK	7.902	1980-12-17	800	[NULL]	20
7.788	SCOTT	ANALYST	7.566	1987-04-19	3.000	[NULL]	20
7.876	ADAMS	CLERK	7.788	1987-05-23	1.100	[NULL]	20
7.698	BLAKE	MANAGER	7.839	1981-05-01	2.850	[NULL]	30
7.499	ALLEN	SALESMAN	7.698	1981-02-20	1.600	300	30
7.521	WARD	SALESMAN	7.698	1981-02-22	1.250	500	30
7.654	MARTIN	SALESMAN	7.698	1981-09-28	1.250	1.400	30
7.844	TURNER	SALESMAN	7.698	1981-09-08	1.500	0	30
7.900	JAMES	CLERK	7.698	1981-12-03	950	[NULL]	30

```
GRANT SELECT(empno, ename, job, mgr, hiredate)  
ON EMP  
TO usr;
```

???

Non está soportado por todos os xestores
(Oracle non o soporta)

Non é posible dar privilexio directamente sobre parte
das filas dunha táboa

Técnicas adicionais de seguridade

Vistas como elemento de seguridade

Solución:

- Retirar os privilexios sobre a táboa (se estaban concedidos).
- Crear unha vista coa consulta desexada. Son comúns:
 - Vista vertical: Selección de todas as filas pero só parte das columnas (atributos).
 - Vista horizontal: Selección de todos os atributos pero só parte das filas.
- Dar permisos sobre a vista. Se é necesario poden usarse roles.

Exemplo:

```
-- Se usr tiña privilexios sobre emp:  
-- REVOKE ALL PRIVILEGES ON emp FROM usr;
```

```
CREATE VIEW emp20  
  AS SELECT empno, ename, job, mgr, hiredate, sal, comm, deptno  
  FROM emp  
  WHERE deptno=20  
  WITH CHECK OPTION;
```

```
GRANT ALL PRIVILEGES ON emp20 to usr;
```

Técnicas adicionais de seguridade

Bases de datos estatísticas

Contén datos individuais, pero só publica datos agregados.

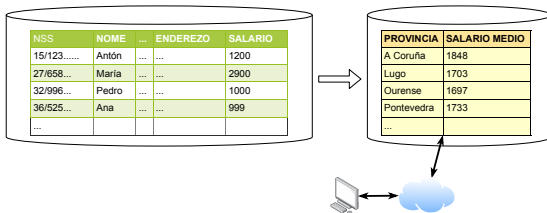
É unha forma de “anonimizar” e non publicar información confidencial.

Exemplos:

- Salarios medios por profesión e zona.
- Datos sanitarios (incidencias COVID por área sanitaria).

Posible problema: a inferencia de datos confidenciais a partir dos agregados.

Habitualmente, non se publicarán datos se non proveñen dun certo número mínimo de tuplas.



Técnicas adicionais de seguridade

Cifrado de información

As comunicacións co SXBD deben estar cifradas, especialmente se o cliente se conecta a través de internet. Úsase habitualmente SSL/TLS e certificados.

Poden cifrarse tamén os datos da BD, xa que hai información que ninguén (nin o DBA) debe coñecer.

Exemplos:

- Contraseñas dos usuarios da BD, ou de usuarios de aplicacións que se almacenen na BD.
- Información confidencial, como un historial clínico.

Pode protexerse a información ante accesos externos (ex: roubo dun ficheiro da BD, posibilidade de obter información do arquivo “crudo”).

Mecanismos:

- Para as contraseñas é común cifrar externamente a información e almacenar só a clave cifrada.
É habitual é un cifrado unidireccional (HASH, SHA-1, MD5, ...). No proceso de login, cófrase a clave introducida e compárase coa almacenada.
- Para o caso de información confidencial como o historial clínico debe usarse un cifrado bidireccional, para poder recuperar a información cifrada.

Oracle ofrece varios mecanismos, como o paquete DBMS_CRYPT (vía código PL/SQL) ou TDE (Transparent Data Encryption, (des)cifrado de forma transparente desde unha columna dunha táboa ata un tablespace completo).

Técnicas adicionais de seguridade

Prevención de inxección de SQL

A inxección de SQL (*SQL injection*) é un problema de seguridade debido fundamentalmente a non controlar de xeito correcto a entrada de datos nos programas que acceden á BD.

A continuación móstrase un exemplo cun programa (mal) codificado en Python.

```
query = "select datanac from persoa where nome = '{}'"
nompers=input('Introduce o nome da persoa:')
# introducimos: Ada Byron
# Parece correcto:
# query.format(nompers) => "select datanac from persoa where nome = 'Ada Byron'"
cursor.execute(query.format(nompers)) # => Ok
# Pero, OLLLO!
# introducimos nome: '; DROP TABLE persoa; --
query.format(nompers) # "select datanac from persoa where nome = '''; DROP TABLE persoa; -- '"
cursor.execute(query.format(nompers)) # => Elimina a táboa persoa
```

Unha posible solución consiste en utilizar parámetros:

```
query = "select datanac from persoa where nome = %(nompers)s"
nompers=input('Introduce o nome da persoa:')
# Introducimos nome: '; DROP TABLE persoa; --
# Buscaría unha persoa chamada: '; DROP TABLE persoa; --
# Non ocorre a inxección SQL
cur.execute(query, {'nompers' : nompers})
```

Técnicas adicionais de seguridade

Backups

Os backups ou copias de seguridade son tamén unha ferramenta fundamental desde o punto de vista da seguridade.

Pode producirse perda de información por varios motivos:

- Borrado (accidental ou intencionado) por parte dun usuario.
- Mal funcionamento do software (SO, SXBD, outros programas, poderían producir, a corrupción dun ficheiro ou sistema de ficheiros).
- Fallo hardware dos dispositivos.

É importante, polo tanto, dispoñer dun plan global de salvagarda do sistema, e realizar probas periódicas para comprobar o seu correcto funcionamento.

- 1 Introducción
- 2 Autenticación e autorización
- 3 Control de acceso discrecional
- 4 Caso particular: Oracle
- 5 Técnicas adicionais
- 6 Auditoría**

Como activos importantes da nosa organización, para protexer os nosos datos utilizamos:

- Seguridade (a priori)
Por exemplo, creando usuarios e roles para asignarlles privilexios
- **Auditoría** da base de datos: Permite **rexistrar información**
 - Dos eventos globais do sistema (arranque, parada, condicións de erro, ...).
 - Das accións que os usuarios fan sobre os obxectos da BD.
- Análise dos logs de auditoría (a posteriori): permite detectar
 - Accesos, ou intentos de acceso, non autorizados.
 - Abuso no acceso a datos.

Auditoría

Que podemos auditar?

Auditoría básica

1. Acceso de usuarios
2. Uso de privilexios de sistema
3. Modificacións do esquema da base de datos
4. Modificacións de datos sensibles

É posible auditar

- Todos os privilexios de sistema
- A nivel de calquera obxecto da base de datos (táboa, vista)
 - Diversas accións: select, delete, insert, update
 - Para intentos con éxito, sin éxito, ou ambos
 - A nivel de usuario individual ou global (todos os usuarios)

DISCUSIÓN: Debemos auditar todo o posible?

1. Oracle audit

- Sentenzas DDL, xestión de usuarios,...
- Sentenzas DML: A nivel de táboa.

2. Triggers de sistema

- Eventos do sistema: arranque-parada da base de datos
- Conexión/desconexión de usuarios
- Modificación do esquema

3. Triggers de inserción, borrado e eliminación

- Poden chegar a nivel de fila e columna
- Só detectan modificacións (non hai trigger para SELECT)
- Aparte de auditoría podemos facer outras cousas (BD Activas)

4. Auditoría detallada (FGA: Fine Grain Auditing)

- Captura accesos de lectura, ata nivel de fila e columna
- Baseado en triggers internos
- Usa o paquete DBMS_FGA (procedementos PL/SQL)
- Define "políticas" (policies) de auditoría

5. Logs do sistema

- Alert logs: rexistran arranque-parada, cambios estruturais (engadir arquivo á BD), ...

Permite auditar

- Todos os permisos que se poden dar (GRANT) a un usuario ou rol.
 1. Auditoría de sentencias (create table, create session, ...)
 2. Auditoría de privilexios (alter user, grant, ...)
 3. Auditoría de obxectos (select en táboas, ...)

- Exemplos:

```
audit alter table;  
audit create session;  
audit alter user;
```

```
noaudit drop table;
```

- Podemos comprobar no catálogo de Oracle o que estamos auditando:

```
select audit_option, success, failure  
  from dba_stmt_audit_opts  
 union  
select audit_option, success, failure  
  from dba_priv_audit_opts;
```

AUDIT_OPTION	SUCCESS	FAILURE
ALTER TABLE	BY ACCESS	BY ACCESS
ALTER USER	BY ACCESS	BY ACCESS
CREATE SESSION	BY ACCESS	BY ACCESS

Auditoría

Oracle Audit: Auditoría de sesiones

```
SQL> select username,  
2 terminal,  
3 action_name,  
4 to_char(timestamp,'DDMMYYYY:HHMISS') timestamp,  
5 to_char(logoff_time,'DDMMYYYY:HHMISS') logoff_time,  
6 returncode  
7 from dba_audit_session;
```

USERNAME	TERMIN	ACTION_N	TIMESTAMP	LOGOFF_TIME	RETURNCODE
SYSMAN		LOGON	23012022:042338		0
PENABAD	SIL	LOGOFF	23012022:042349	23012022:042739	0
...					
NON_EXISTE	SIL	LOGON	23012022:042855		1017

Auditoría

Triggers de sistema

Exemplo: creación de triggers para registrar acceso á BD.

```
CREATE TABLE CONEXIONS
(
    USUARIO CHAR(20),
    DATA_HORA TIMESTAMP,
    TIPO CHAR(12)
);

CREATE TRIGGER LOG_CONEXION
    AFTER LOGON ON DATABASE
BEGIN
    INSERT INTO CONEXIONS
        VALUES (USER, SYSDATE, 'CONEXIÓN');
END LOG_CONEXION;
/

CREATE TRIGGER LOG_DESCONEXION
    BEFORE LOGOFF ON DATABASE
BEGIN
    INSERT INTO CONEXIONS
        VALUES (USER, SYSDATE, 'DESCONEXIÓN');
END LOG_CONEXION;
/
```

Exemplo: análise do “log” creado polos triggers.

```
SQL> select * from conexions;
```

ninguna fila seleccionada

```
SQL> disconnect  
[Desconectado]
```

```
SQL> connect penabad/*****  
Conectado.  
SQL> select * from conexions;
```

USUARIO	DATA_HORA	TIPO
-----	-----	-----
PENABAD	31/01/22 16:47:41,000000	DESCONEXIÓN
PENABAD	31/01/22 16:47:44,000000	CONEXIÓN

Auditoría

Triggers de inserción, borrado e modificación

Exemplo: queremos auditar os cambios (update) dos salarios dos empregados.

```
CREATE OR REPLACE TRIGGER T_LOG_CAMBIO_SAL
AFTER UPDATE OF SAL ON EMP
FOR EACH ROW
BEGIN
    INSERT INTO LOGCAMBIOSAL
        (USUARIO,TIMESTAMP,EMPNO,SAL_ANT,SAL_NOVO)
    VALUES
        (USER, SYSDATE, :NEW.EMPNO, :OLD.SAL, :NEW.SAL);
END;
```

Que podemos deducir deste contido do log?

USUARIO	TIMESTAMP	EMPNO	SAL_ANT	SAL_NOVO
SCOTT	23/01/2022 08:45:40,000000	7369	800	8
SCOTT	23/01/2022 08:46:22,000000	7369	8	800

Exemplo: quedarán rexistrados os accesos á táboa DEPT do usuario SCOTT, sempre que se acceda á columna DNAME e o valor de DEPTNO sexa 30.

```
begin
  dbms_fga.add_policy (
    object_schema=>'SCOTT',
    object_name=>'DEPT',
    policy_name=>'nome_ventas',
    audit_column => 'DNAME',
    audit_condition => 'deptno = 30' );
end;

select timestamp, db_user, object_name, sql_text
  from dba_fga_audit_trail;
```

TIMESTAMP	DB_USER	OBJECT_NAME	SQL_TEXT
15/01/20	PENABAD	DEPT	select dname from dept
15/01/20	PENABAD	DEPT	select * from dept
15/01/20	PENABAD	DEPT	select loc from dept where dname='SALES'

As seguintes consultas non se rexistran, por non cumprir as condicións da política.

```
select loc from dept where deptno=10;
select deptno, loc from dept;
```

Bibliografía

- [1] Ramez A. Elmasri e Shamkant B. Navathe. *Fundamentos de sistemas de bases de datos*. 5ª ed. Addison Wesley, 2007.
- [2] Pete Finnigan. *Introduction to Simple Oracle Auditing*.
<http://www.securityfocus.com/print/infocus/1689>. Última visita: xaneiro 2006.
- [3] Hector Garcia-Molina, Jeffrey D. Ullman e Jennifer Widom. *Database Systems: The Complete Book*. 2ª ed. Upper Saddle River, NJ, USA: Prentice Hall Press, 2008.
- [4] J. Melton e A. R. Simon. *SQL:1999 - Understanding relational language components*. Morgan Kaufmann, 2002. ISBN: 1558604561.